Terminal and HPC 101

Dewangga Alfarisy PhD in Civil Engineering dewangga.alfarisy@glasgow.ac.uk

Table of contents

1 Foreword **1**

2 What is HPC? **1**

3 Why do we use HPC? **2**

4 Typical Components of an HPC System **2**

5 How Users Access an HPC Cluster **2**

6 Basic Linux and Terminal Skills for HPC **3**

6.1 File System Navigation . . . 3
6.2 File Management . . . 5
6.3 Adding texts to file . . . 9
6.4 Viewing File Contents . . . 11
6.5 Running Programs . . . 14

1 Foreword

High-Performance Computing (HPC) refers to the use of powerful computers that combine many processors to solve problems much faster than a typical personal computer. Instead of relying on a single CPU, HPC systems distribute work across many CPUs or GPUs, allowing complex computations to be performed efficiently.

HPC is widely used in science, engineering, medicine, and data-intensive research. For example, engineers use HPC to run large simulations, process large datasets, train machine learning models, and perform advanced optimisation or uncertainty analysis.

In universities, HPC systems are shared resources that researchers and students access remotely. Because these systems are typically operated through the command line, understanding basic Linux commands and terminal usage is an essential first step to working effectively on an HPC cluster.

2 What is HPC?

High-Performance Computing (HPC) uses many interconnected processors to solve computational problems faster than a standard computer. These systems are often organised as **clusters**, where multiple machines work together to perform tasks in parallel.

By distributing workloads across many processors, HPC systems can handle large simulations, data analysis tasks, and computational models that would otherwise take too long to run on a personal computer.

3 Why do we use HPC?

Many scientific and engineering problems require more computational power than a typical laptop or desktop can provide. HPC enables researchers to perform large-scale calculations efficiently.

Common reasons for using HPC include:

- Running **large simulations** such as fluid dynamics or structural modelling
- **Reducing computation time** through parallel processing
- Handling **large datasets** in scientific or data-driven research
- Training **machine learning and AI models** that require significant computing resources

4 Typical Components of an HPC System

An HPC cluster typically includes several key components:

Login Nodes

Login nodes are the entry point to the cluster. Users connect to these nodes remotely to manage files, prepare scripts, and submit jobs.

Compute Nodes

Compute nodes perform the actual computations. Jobs submitted by users are scheduled to run on these machines.

Storage Systems

Shared storage allows users to access data and results from any node in the cluster.

Job Scheduler

A scheduler distributes jobs across available resources such as CPUs, memory, and GPUs. Common schedulers include **SLURM**, **PBS**, and **LSF**.

5 How Users Access an HPC Cluster

Users typically access an HPC cluster remotely using **Secure Shell (SSH)**. After logging in, they interact with the system through a terminal to manage files, load software modules, and submit jobs to the scheduler.

To connect to the HPC cluster, open a terminal application. This can be:

- **Terminal** on macOS or Linux
- **Command Prompt** or **PowerShell** on Windows

Then run the following command:

```
ssh <id>@cognition.cose.gla.ac.uk
```

where `<id>` should be replaced with your university or HPC username.

```
(base) dalfarisy@Dewanggas-MacBook-Pro ~ % ssh 2949472a@cognition.cose.gla.ac.uk

2949472a@cognition.cose.gla.ac.uk's password:
```

Figure 1: Example SSH login to the HPC cluster

After running the command, you will be prompted to enter your password. While typing the password, **no characters or cursor movement will appear on the screen**. This is normal behaviour in Linux systems for security reasons. Simply type your password carefully and press **Enter**.

During the first login, you may be asked to reset your password. If prompted:

1. Enter your **current password** (the one used to log in).
2. Enter a **new password**.
3. Re-enter the new password to confirm.

Once the password is successfully updated, you will be logged out of the HPC system. Simply put again the ssh sign and re-login with the new password.

6 Basic Linux and Terminal Skills for HPC

Most HPC systems run on Linux and are operated through the command line (Command Prompt for Windows, Terminal for Mac and Linux). Therefore, basic familiarity with Linux commands is essential.

Key skills include:

6.1 File System Navigation

When working on an HPC system, most interaction happens through the **file system**.
Understanding how to check your location and move between directories is essential.

6.1.1 Check Your Current Location (`pwd`)

The command `pwd` stands for **print working directory**. It tells you where you currently are in the file system.

Example:

```
pwd
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ pwd
/users/2949472a
```

Figure 2: Location checking using pwd

This means you are currently inside your **home directory**.

It is good practice to run `pwd` when you are unsure where you are in the file system.

6.1.2 List Files and Folders (`ls`)

The `ls` command shows the contents of the current directory.

Example:

```
ls
```

Example output:

Figure 3: List files and folders

Other significant command is to add -l for showing additional informations.

Example:

```
ls -l
```

Example output:

Figure 4: List of Files and Folders with Additional Informations

This shows additional information such as:

- file permissions
- file size
- modification time

6.1.3 Change Directory (`cd`)

The `cd` command allows you to move between directories.

Example:

```
cd Test
```

Example output:

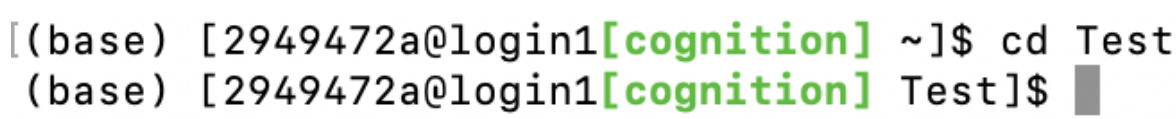


Figure 5: Moving directories

6.2 File Management

File management allows users to create directories, copy files, move files, and remove files within the HPC system. These commands are essential for organising data and managing project files.

The most commonly used commands are:

- `mkdir` – create a new directory
- `cp` – copy files or directories
- `mv` – move or rename files
- `rm` – remove files or directories
- `sftp` – transfer files between your computer and the HPC system

6.2.1 Create a Directory (`mkdir`)

The `mkdir` command creates a new folder. The `nano` command create a new text file for our tests.

```
mkdir newFolder
nano text1.txt
```

When using the nano command, you will encounter movement to different UI, dont worry and for now, just exit the UI.

You can confirm the new folder was created by

```
ls
```

Example output:

```
[(base) [2949472a@login1[cognition] ~]$ ls
[archive  localscratch  newFolder  sharedscratch  Test
[(base) [2949472a@login1[cognition] ~]$ nano text1.txt
[(base) [2949472a@login1[cognition] ~]$ ls
 archive  localscratch  newFolder  sharedscratch  Test  text1.txt
```

Figure 6: Making new directory

6.2.2 Copy Files (`cp`)

The `cp` command copies a file from one location to another.

Example:

```
cp text1.txt text2.txt
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ cp text1.txt text2.txt
(base) [2949472a@login1[cognition] ~]$ ls
archive  localscratch  newFolder  sharedscratch  Test  text1.txt  text2.txt
```

Figure 7: Copy files to different file name

This creates a copy of `text1.txt` called `text2.txt`.

You can also copy a file into a directory.

Example:

```
cp text1.txt newFolder/
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ cp text1.txt newFolder/
(base) [2949472a@login1[cognition] ~]$ ls
archive  localscratch  newFolder  sharedscratch  Test  text1.txt  text2.txt
(base) [2949472a@login1[cognition] ~]$ cd newFolder
(base) [2949472a@login1[cognition] newFolder]$ ls
text1.txt
```

Figure 8: Copy files to folder

This creates a copy of `text1.txt` inside `newFolder`.

6.2.3 Move or Rename Files (`mv`)

The `mv` command moves files between directories or renames them.

Rename a file:

```
mv text1.txt newfile.txt
```

Example output:

```
(base) [2949472a@login1[cognition] newFolder]$ mv text1.txt newfile.txt
(base) [2949472a@login1[cognition] newFolder]$ ls
newfile.txt
```

Figure 9: Rename files

This renames `text1.txt` to `newfile.txt`

Move a file to directory:

```
mv text2.txt newFolder/
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ mv text2.txt newFolder/
(base) [2949472a@login1[cognition] ~]$ cd newFolder
(base) [2949472a@login1[cognition] newFolder]$ ls
newfile.txt  text2.txt
```

Figure 10: Moving files to folder

This moves `text2.txt` to `newFolder`

6.2.4 Remove Files (`rm`)

The `rm` command removes files.

Example:

```
rm text1.txt
```

Example output:

```
(base) [2949472a@login1[cognition] newFolder]$ rm text2.txt
(base) [2949472a@login1[cognition] newFolder]$ ls
newfile.txt
```

Figure 11: Remove files

This removes `text2.txt`.

To remove a folder, add -r to the given command.

Example:

```
mkdir removable-folder
ls
rm -r removable-folder
```

Example output:

```
(base) [2949472a@login1[cognition] newFolder]$ mkdir removable-folder
(base) [2949472a@login1[cognition] newFolder]$ ls
newfile.txt  removable-folder
(base) [2949472a@login1[cognition] newFolder]$ rm -r removable-folder
(base) [2949472a@login1[cognition] newFolder]$ ls
newfile.txt
```

Figure 12: Remove folder

This removes `removable-folder` and all corresponding files inside.

Important:
Files deleted with `rm` are permanently removed.
There is **no recycle bin or undo option** on most HPC systems.

Always confirm your location before deleting files:

```
pwd
```

6.2.5 Transfer Files Using `sftp`

Files can be transferred between your local computer and the HPC cluster using `sftp`.

Start an SFTP session:

```
sftp <id>@cognition.cose.gla.ac.uk
```

Once connected, you can move your files:

```
put <file directory>
```

As an example, we want to put the helloworld.ipnyb in our training-files to the hpc system from our local computer.

Example code:

```
put <file directory>/hello-world.ipynb
```

Example output:

```
sftp> put /Users/dalfarisy/Library/CloudStorage/OneDrive-UniversityofGlasgow/PhD
-related/quantum-ml-pqc/uog-hpc/training-files/hello-world.ipynb
Uploading /Users/dalfarisy/Library/CloudStorage/OneDrive-UniversityofGlasgow/PhD
-related/quantum-ml-pqc/uog-hpc/training-files/hello-world.ipynb to /users/29494
72a/hello-world.ipynb
hello-world.ipynb                             100%  255    42.0KB/s   00:00
```

Figure 13: Practice putting files using SFTP.

Or download your file from HPC:

As an example, we want to download the same file into our local computer.

```
get <file/folder in hpc> <file/folder directory>
```

Example output:

```
sftp> get hello-world.ipynb /Users/dalfarisy/Documents
Fetching /users/2949472a/hello-world.ipynb to /Users/dalfarisy/Documents/hello-w
orld.ipynb
hello-world.ipynb                             100%  255    22.9KB/s   00:00
```

Figure 14: Practice downloading files using SFTP.

6.2.6 Practice Example

Here are simple practice example for familiarise yourself with Linux system.

Example code:

```
mkdir testfolder
nano text1.txt
ls
cp text1.txt testfolder/
mv text1.txt file_backup.txt
ls
rm file_backup.txt
ls
cd testfolder
ls
```

Example output:

```
(base) [2949472a@login1[cognition] newFolder]$ mkdir testfolder
(base) [2949472a@login1[cognition] newFolder]$ nano text1.txt
(base) [2949472a@login1[cognition] newFolder]$ ls
testfolder  text1.txt
(base) [2949472a@login1[cognition] newFolder]$ cp text1.txt testfolder/
(base) [2949472a@login1[cognition] newFolder]$ mv text1.txt file_backup.txt
(base) [2949472a@login1[cognition] newFolder]$ ls
file_backup.txt  testfolder
(base) [2949472a@login1[cognition] newFolder]$ rm file_backup.txt
(base) [2949472a@login1[cognition] newFolder]$ ls
testfolder
(base) [2949472a@login1[cognition] newFolder]$ cd testfolder
(base) [2949472a@login1[cognition] testfolder]$ ls
text1.txt
(base) [2949472a@login1[cognition] testfolder]$
```

Figure 15: Practice example code and its given output

These commands will help you practise creating directories, copying files, moving files, and removing files.

In order to use this training, we upload hpc-text.txt that is shared in this github using SFTP by using:

```
sftp <id>@cognition.cose.gla.ac.uk
put <directory>/hpc-text.txt
```

6.3 Adding texts to file

6.3.1 Using echo to create files

We can use echo to create text files,

Example:

```
echo "This is a test"
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ echo "This is a test."
This is a test.
```

Figure 16: Echo some words

Yes, echo just prints its arguments back out again (hence the name). But combine it with a redirect, and you've got a way to easily create small test files:

Now we are saving the words we put in echo as a txt file.

Example:

```
echo "This is a test" > text1.txt
echo "This is a second test" > text2.txt
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ echo "This is a test." > test1.txt
(base) [2949472a@login1[cognition] ~]$ echo "This is a second test." > test2.txt
```

Figure 17: Echo saving the words as txt

We also can link both files together as one, this is called concatenate. The first two combined will concatenate the file together and the last > will act as saving to a new file.

Example:

```
cat text_1.txt text_2.txt > cat.txt
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ cat test1.txt test2.txt > cat.txt
```

Figure 18: Concatenate using cat to combine both files as new txt

6.3.2 Using nano to modify file

Using nano is much more straightforward as we can directly edit the file in the nano ecosystem and we can do various things on this environment too. We can directly edit, modify and save the files using this software.

```
nano test_1.txt
```

Example output:

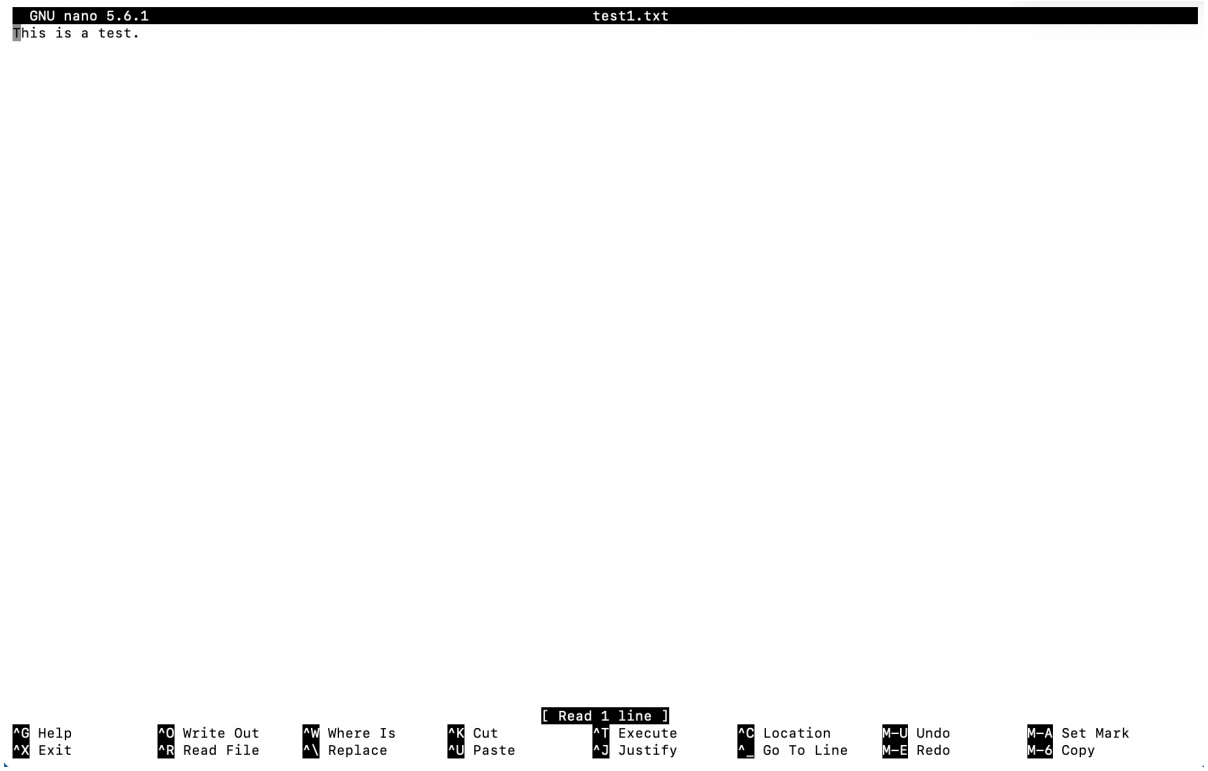


Figure 19: Nano user interface in terminal.

6.4 Viewing File Contents

Sometimes it is useful to inspect the contents of a file directly from the terminal without opening a full text editor. Linux provides several commands that allow users to quickly view files.

Common commands include:

- `cat` – display the entire file
- `less` – view a file page by page
- `head` – show the first lines of a file
- `tail` – show the last lines of a file

6.4.1 Display an Entire File (`cat`)

The `cat` command prints the full contents of a file directly in the terminal.

```
cat hpc-text.txt
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ cat hpc-text.txt
The Rise of High-Performance Computing

 High-performance computing (HPC) refers to the use of powerful computer systems designed to process extremely large volumes of data and per
form complex calculations at very high speeds. Unlike conventional desktop computers, HPC systems typically combine hundreds or thousands o
f processors that work together to solve scientific, engineering, and data-intensive problems. This capability has transformed modern resea
rch by enabling simulations and analyses that were previously impossible.

One of the most significant advantages of HPC is its ability to perform parallel computation. In traditional computing, tasks are often exe
cuted sequentially, meaning each calculation must wait for the previous one to finish. In contrast, HPC systems divide large problems into
 smaller tasks that can be processed simultaneously across many computing units. This parallel approach dramatically reduces the time requir
ed to complete large calculations.

 HPC has become an essential tool in a wide range of disciplines. In climate science, researchers use HPC to simulate atmospheric and ocean
systems in order to understand long-term climate patterns. These simulations involve enormous datasets and require sophisticated mathematic
al models. Without HPC systems, running such simulations could take years rather than days or weeks.

Similarly, engineers rely on HPC to perform advanced simulations of physical systems. For example, aerospace engineers use computational mo
dels to study airflow around aircraft structures, while civil engineers simulate stress distribution in bridges and buildings. By analysing
 these models on HPC clusters, researchers can test designs under many different conditions before constructing physical prototypes.

Another rapidly growing application of HPC is in artificial intelligence and machine learning. Training large machine learning models often
 requires processing millions or even billions of data points. Graphics processing units (GPUs) and other accelerators in HPC clusters allo
w these models to be trained far more efficiently than on standard computers.

Despite these benefits, working with HPC systems requires specialised knowledge. Users must typically interact with the system through a co
mmand-line interface and submit jobs to a scheduler that manages the allocation of computational resources. This workflow is different from
 the familiar graphical interfaces used on personal computers, and new users often need training to become comfortable with it.

Most HPC clusters are shared environments used by many researchers simultaneously. To ensure fair access, resource management software sche
dules computational jobs and distributes workloads across available machines. Systems such as SLURM, PBS, and LSF are commonly used for thi
s purpose. By controlling how resources are allocated, these schedulers prevent individual users from monopolising the system.

Looking ahead, the importance of HPC is expected to grow even further. As scientific questions become more complex and datasets continue to
 expand, researchers will increasingly depend on powerful computing infrastructure to support discovery and innovation. In fields ranging f
rom medicine to astrophysics, HPC is likely to remain a central component of m(base) [2949472a@log(base) [2949472a@login1[cognition] ~(base
```

Figure 20: Viewing entire file using cat

This command works well for **small files**, but it may flood the screen if the file is very large.

6.4.2 View Files Page by Page (`less`)

The `less` command allows you to scroll through a file interactively.

Inside `less`, you can:

- press **Space** to scroll down
- press **b** to scroll up
- press **q** to quit

This is the preferred command for viewing **large files**.

Example:

```
less hpc-text.txt
```

Example output:

```
The Rise of High-Performance Computing

High-performance computing (HPC) refers to the use of powerful computer systems designed to process extremely large volumes of data and per
form complex calculations at very high speeds. Unlike conventional desktop computers, HPC systems typically combine hundreds or thousands o
f processors that work together to solve scientific, engineering, and data-intensive problems. This capability has transformed modern resea
rch by enabling simulations and analyses that were previously impossible.

One of the most significant advantages of HPC is its ability to perform parallel computation. In traditional computing, tasks are often exe
cuted sequentially, meaning each calculation must wait for the previous one to finish. In contrast, HPC systems divide large problems into
smaller tasks that can be processed simultaneously across many computing units. This parallel approach dramatically reduces the time requir
ed to complete large calculations.

HPC has become an essential tool in a wide range of disciplines. In climate science, researchers use HPC to simulate atmospheric and ocean
systems in order to understand long-term climate patterns. These simulations involve enormous datasets and require sophisticated mathematic
al models. Without HPC systems, running such simulations could take years rather than days or weeks.

Similarly, engineers rely on HPC to perform advanced simulations of physical systems. For example, aerospace engineers use computational mo
dels to study airflow around aircraft structures, while civil engineers simulate stress distribution in bridges and buildings. By analysing
 these models on HPC clusters, researchers can test designs under many different conditions before constructing physical prototypes.

Another rapidly growing application of HPC is in artificial intelligence and machine learning. Training large machine learning models often
 requires processing millions or even billions of data points. Graphics processing units (GPUs) and other accelerators in HPC clusters allo
w these models to be trained far more efficiently than on standard computers.

Despite these benefits, working with HPC systems requires specialised knowledge. Users must typically interact with the system through a co
mmand-line interface and submit jobs to a scheduler that manages the allocation of computational resources. This workflow is different from
 the familiar graphical interfaces used on personal computers, and new users often need training to become comfortable with it.

Most HPC clusters are shared environments used by many researchers simultaneously. To ensure fair access, resource management software sche
dules computational jobs and distributes workloads across available machines. Systems such as SLURM, PBS, and LSF are commonly used for thi
s purpose. By controlling how resources are allocated, these schedulers prevent individual users from monopolising the system.

Looking ahead, the importance of HPC is expected to grow even further. As scientific questions become more complex and datasets continue to
 expand, researchers will increasingly depend on powerful computing infrastructure to support discovery and innovation. In fields ranging f
rom medicine to astrophysics, HPC is likely to remain a central component of modern scientific progress.
hpc-text.txt (END)
```

Figure 21: Viewing entire file using less

6.4.3 View the Beginning of a File (`head`)

The `head` command shows the **first 10 lines** of a file.

Example:

```
head hpc-text.txt
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ head hpc-text.txt
The Rise of High-Performance Computing

High-performance computing (HPC) refers to the use of powerful computer systems designed to process extremely large volumes of data and per
form complex calculations at very high speeds. Unlike conventional desktop computers, HPC systems typically combine hundreds or thousands o
f processors that work together to solve scientific, engineering, and data-intensive problems. This capability has transformed modern resea
rch by enabling simulations and analyses that were previously impossible.

One of the most significant advantages of HPC is its ability to perform parallel computation. In traditional computing, tasks are often exe
cuted sequentially, meaning each calculation must wait for the previous one to finish. In contrast, HPC systems divide large problems into
smaller tasks that can be processed simultaneously across many computing units. This parallel approach dramatically reduces the time requir
ed to complete large calculations.

HPC has become an essential tool in a wide range of disciplines. In climate science, researchers use HPC to simulate atmospheric and ocean
systems in order to understand long-term climate patterns. These simulations involve enormous datasets and require sophisticated mathematic
al models. Without HPC systems, running such simulations could take years rather than days or weeks.

Similarly, engineers rely on HPC to perform advanced simulations of physical systems. For example, aerospace engineers use computational mo
dels to study airflow around aircraft structures, while civil engineers simulate stress distribution in bridges and buildings. By analysing
 these models on HPC clusters, researchers can test designs under many different conditions before constructing physical prototypes.
```

Figure 22: Viewing beginning of a file using head

You can specify the number of lines:

```
head -n 5 file.txt
```

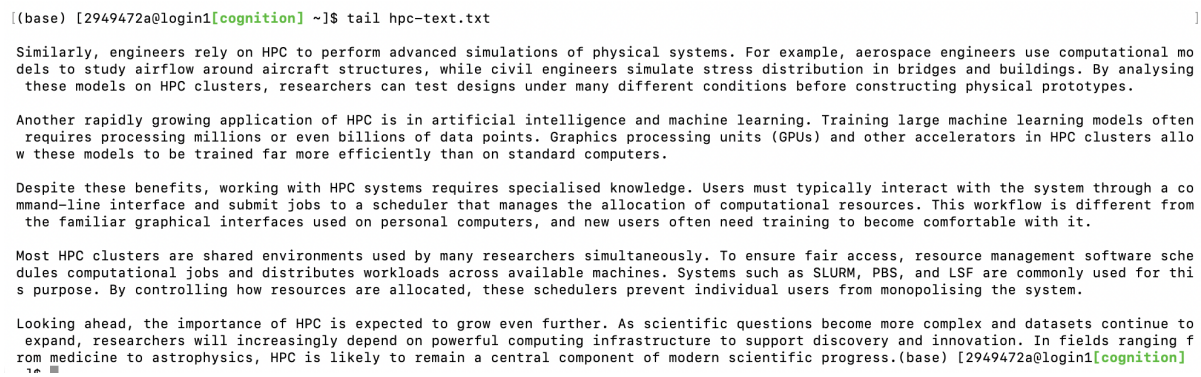
6.4.4 View the End of a File (`tail`)

The `tail` command shows the **last 10 lines** of a file.

Example:

```
tail hpc-text.txt
```

Example output:



```
(base) [2949472a@login1[cognition] ~]$ tail hpc-text.txt

Similarly, engineers rely on HPC to perform advanced simulations of physical systems. For example, aerospace engineers use computational mo
dels to study airflow around aircraft structures, while civil engineers simulate stress distribution in bridges and buildings. By analysing
 these models on HPC clusters, researchers can test designs under many different conditions before constructing physical prototypes.

Another rapidly growing application of HPC is in artificial intelligence and machine learning. Training large machine learning models often
 requires processing millions or even billions of data points. Graphics processing units (GPUs) and other accelerators in HPC clusters allo
w these models to be trained far more efficiently than on standard computers.

Despite these benefits, working with HPC systems requires specialised knowledge. Users must typically interact with the system through a co
mmand-line interface and submit jobs to a scheduler that manages the allocation of computational resources. This workflow is different from
 the familiar graphical interfaces used on personal computers, and new users often need training to become comfortable with it.

Most HPC clusters are shared environments used by many researchers simultaneously. To ensure fair access, resource management software sche
dules computational jobs and distributes workloads across available machines. Systems such as SLURM, PBS, and LSF are commonly used for thi
s purpose. By controlling how resources are allocated, these schedulers prevent individual users from monopolising the system.

Looking ahead, the importance of HPC is expected to grow even further. As scientific questions become more complex and datasets continue to
 expand, researchers will increasingly depend on powerful computing infrastructure to support discovery and innovation. In fields ranging f
rom medicine to astrophysics, HPC is likely to remain a central component of modern scientific progress.(base) [2949472a@login1[cognition]
```

Figure 23: Viewing end of file using tail

You can specify the number of lines:

```
tail -n 5 file.txt
```

These commands allow you to quickly inspect files without opening a text editor.

One key example is to take the last data in the log file, which is coming quite handy in some analysis:

```
tail -f output.log
```

6.4.5 Practice Exercise

Try the following commands:

```
cat hpc-text.txt
less hpc-text.txt
head hpc-text.txt
tail hpc-text.txt
```

These commands allow you to quickly inspect files without opening a text editor and feel the difference between each methods.

6.5 Running Programs

After navigating files and managing directories, the next step in using an HPC system is running programs. Programs can be executed directly from the terminal or submitted as jobs to the scheduler.

In general:

- small tests or simple scripts can be run directly in the terminal
- larger computations should be submitted to the scheduler

6.5.1 Running a Script Directly using python

First, we create a simple script using nano. The file should directly printing “Hello world!” as a simple example.

Example:

```
nano testrun.py
```

Example output:

Figure 24: Making new py files.

In nano user interface, we use simple command to print “Hello world!”.

Example:

```
print("Hello world!")
```

Example output:

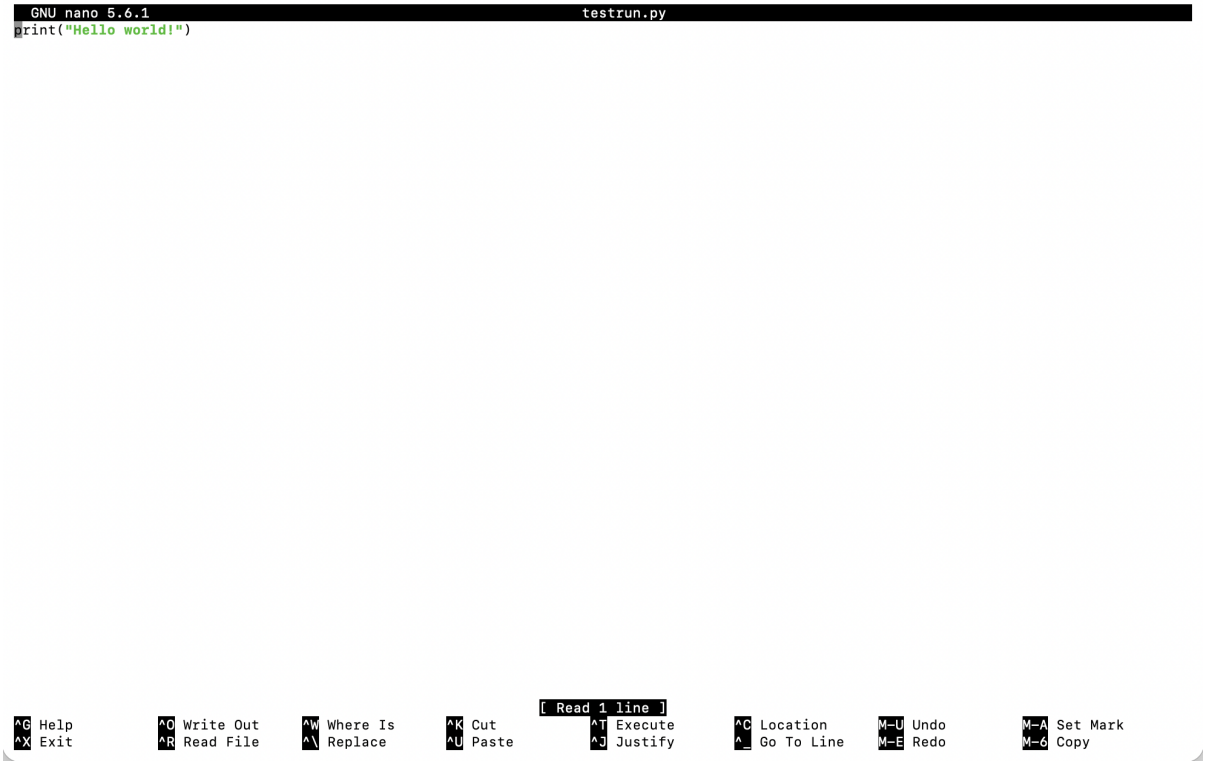


Figure 25: Writing up hello world in nano.

Then, we can check the hpc system if they have a python installed and we can use their python. We can check the module by using the following code

Example:

```
module avail
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ module avail

------------------------------------------------------- /opt/apps/eb/modules/compiler -------------------------------------------------------
   GCC/15.2.0    GCCcore/15.2.0

------------------------------------------------------- /opt/apps/eb/modules/devel ----------------------------------------------------------
   M4/1.4.19    M4/1.4.20-GCCcore-15.2.0    M4/1.4.20 (D)

------------------------------------------------------- /opt/apps/eb/modules/lang -----------------------------------------------------------
   Bison/3.8.2-GCCcore-15.2.0    Bison/3.8.2 (D)    flex/2.6.4-GCCcore-15.2.0    flex/2.6.4 (D)

------------------------------------------------------- /opt/apps/eb/modules/lib ------------------------------------------------------------
   zlib/1.3.1-GCCcore-15.2.0    zlib/1.3.1 (D)

------------------------------------------------------- /opt/apps/eb/modules/tools ----------------------------------------------------------
   EasyBuild/5.1.2    binutils/2.45-GCCcore-15.2.0    binutils/2.45 (D)    help2man/1.49.3-GCCcore-15.2.0

----------------------------------------- /opt/gridware/subscribed/etc/modulefiles/proprietary -----------------------------------------
   ansys/2025.2    comsol/6.3    matlab/r2025a

------------------------------------- /opt/gridware/subscribed/etc/modulefiles/linux-rocky9-x86_64_v4 -------------------------------------
   apptainer/1.4.4/gcc-15.2.0/a-a2wyui6        gcc/15.2.0/gcc-11.5.0/a-mffqiek
   cmake/3.31.9/gcc-11.5.0/a-xc6da5e           lmod/8.7.55/gcc-11.5.0/a-sntlpdr
   cuda/12.9.1/none-none/a-j4amwzt             miniforge3/25.3.0-3/none-none/a-4lhn6xy
   cuda/13.0.2/none-none/a-eai3dlo      (D)    openfoam/2412/gcc-15.2.0/openmpi-5.0.8/a-xb2i6l4
   easybuild/4.7.0/none-none/a-j3rz2ko         r/4.5.2/gcc-15.2.0/a-lxuwftt
```

Figure 26: Loading up available module

In the cognition, we have miniforge so we can load the miniforge. Copy the file names of the miniforge like the one below (updated 17/3/2026)

Example:

```
module load miniforge3/25.3.0-3/none-none/a-4lhn6xy
```

Example output:

```
[2949472a@login1[cognition] ~]$ module load miniforge3/25.3.0-3/none-none/a-4lhn6xy
[2949472a@login1[cognition] ~]$
```

Figure 27: Loading up miniforge module

From here on, we can make conda environment as usual, for simplicity, we will use the base.

Example:

```
conda activate base
```

Example output:

```
[2949472a@login1[cognition] ~]$ module load miniforge3/25.3.0-3/none-none/a-4lhn6xy
[2949472a@login1[cognition] ~]$
```

Figure 28: Loading up conda environment.

From here, we can run the python3 file as usual.

Example:

```
python testrun.py
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ python testrun.py
Hello world!
```

Figure 29: Running python testrun.py file.

However, when trying to run the python notebook (ipynb files), we need to convert it first to a standard python notebook.

Example:

```
jupyter nbconvert --to python hello-world.ipynb
python hello-world.py
```

Example output:

```
(base) [2949472a@login1[cognition] ~]$ jupyter nbconvert --to python hello-world.ipynb
[NbConvertApp] Converting notebook hello-world.ipynb to python
[NbConvertApp] Writing 87 bytes to hello-world.py
(base) [2949472a@login1[cognition] ~]$ python hello-world.py
Hello world!
```

Figure 30: Converting and running jupyter notebook (ipynb) file.